

Serveur de fichiers Samba sécurisé

Du groupe Unix au partage réseau : isoler 4 départements,
de zéro, sans tricher sur la sécurité.

AUTEUR	Rachidi Tchêgoun AGRO
FORMATION	ENSGMM / UNSTIM — Génie Mathématique et Modélisation, Abomey, Bénin
TRACK	Cybersécurité autodidacte — Programme Phoenix
STACK	Docker · Ubuntu Server · Samba 4.19 · Bash · ACL POSIX

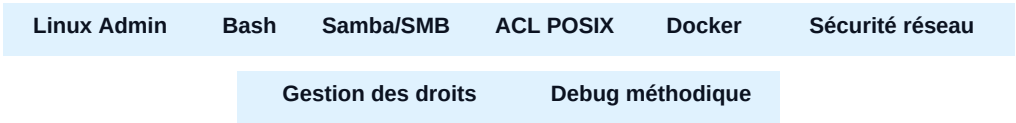
Le pitch

Une entreprise fictive — **4 départements** (RH, IT, Finance, Direction), **11 employés** — a besoin d'un espace réseau où chacun accède à SES fichiers, et rien que ceux-là. Pas de coin de table où IT fouille dans les payes de la Direction. Objectif : construire ce serveur de A à Z, comprendre chaque brique, et être capable de reproduire ça sur un vrai poste en entreprise, sans filet.

Ce document retrace la démarche complète : de la norme FHS jusqu'au test réseau réel avec un client SMB. Chaque section correspond à une couche de sécurité — *et chaque couche s'appuie sur la précédente.*

Compétences mobilisées

- Administration Linux : utilisateurs, groupes, gestion des processus/services
- Modèle de permissions POSIX (rwx, octal) et ACL avancées
- Norme FHS (Filesystem Hierarchy Standard)
- Bash scripting : boucles, tableaux, heredoc, chaînage de commandes
- Configuration et administration du service Samba (SMB/CIFS)
- Méthodologie de test — validation systématique avant mise en service
- Environnement conteneurisé (Docker) pour lab reproductible



Architecture cible

Avant de taper la moindre commande : on pose le plan. Quatre groupes, quatre dossiers isolés, un point d'entrée commun mais neutre.

Service	Groupe Unix	Chemin	Utilisateurs
RH	rh	/srv/samba/rh	rh_awa, rh_anne, rh_armand
IT	it	/srv/samba/it	it_admin, it_senior, it_junior
Finance	finance	/srv/samba/finance	finance_bio, finance_ben, finance_boris
Direction	direction	/srv/samba/direction	direction_dg, direction_pdg

Pourquoi /srv et pas /home ? Parce que la norme FHS réserve /home aux données personnelles d'un compte, alors que /srv est le dossier prévu pour les ressources *servies* à d'autres — exactement notre cas de figure.

Couche 1 — La fondation système

Rien ne fonctionne côté réseau si la base locale est bancale. On construit d'abord l'isolation **sur la machine elle-même**, avant même de parler réseau.

Création des groupes et de l'arborescence

```
groupadd rh && groupadd it && groupadd finance && groupadd direction
mkdir -p /srv/samba/{rh,it,finance,direction}
```

Verrouillage par service

```
chown :rh /srv/samba/rh && chmod 770 /srv/samba/rh
chown :it /srv/samba/it && chmod 770 /srv/samba/it
# ... répété pour finance et direction
```

770 = propriétaire et groupe ont un accès complet, « **autres** » n'ont strictement rien. Un membre du groupe finance ne peut même pas savoir si le dossier RH contient un fichier.

Le piège classique : la permission de traversée

Premier vrai blocage du projet — et probablement le concept le plus mal compris chez les débutants. Un utilisateur RH bien configuré, dans le bon groupe, avec les bonnes permissions sur son dossier... se voit quand même refuser l'accès :

```
rh_awa@srv:~$ cd /srv/samba/rh
-bash: cd: /srv/samba/rh: Permission denied
```

La raison : le dossier **parent** /srv/samba bloquait le passage. Sous Linux, atteindre un fichier exige la permission **x** (traverser) sur *chaque* dossier du chemin — pas seulement sur la destination finale. Et comme un dossier n'a qu'un seul groupe propriétaire à la fois, impossible de satisfaire 4 groupes différents avec un simple **chmod**.

La solution : les ACL

Les ACL POSIX permettent d'empiler plusieurs règles indépendantes sur un même dossier, sans toucher au propriétaire :

```
setfacl -m g:rh:x /srv/samba  
setfacl -m g:it:x /srv/samba  
setfacl -m g:finance:x /srv/samba  
setfacl -m g:direction:x /srv/samba
```

Résultat : les 4 groupes peuvent désormais *traverser* le dossier parent — sans pouvoir le lister (`ls`) ni écrire dedans. Chacun garde un accès strict à son seul sous-dossier.

Couche 2 — Le pont réseau (Samba)

Jusqu'ici, tout marche uniquement en local ([su](#) - [utilisateur](#)). Rien n'est encore visible depuis une autre machine. Samba est la couche qui transforme ce travail Linux en ressource accessible sur le réseau, via le protocole SMB.

Deux identités par utilisateur

Compte système (useradd)	Compte Samba (smbpasswd)
Existe sur la machine	Existe pour le réseau (SMB)
Stocké dans /etc/shadow	Base séparée, hash NTLM
Prérequis obligatoire	Ne peut pas exister sans le 1er

Création en masse des comptes Samba, via une boucle bash (et pas 11 commandes copiées-collées) :

```
liste_des_users=(rh_anne rh_armand it_admin it_senior it_junior \  
  finance_bio finance_boris finance_ben direction_dg direction_pdg)  
  
for user in "${liste_des_users[@]}"  
do  
    echo -e "$user@123\n$user@123" | smbpasswd -a -s $user  
done
```

La configuration : smb.conf

Chaque partage relie un dossier physique à un groupe autorisé :

```
[rh]  
path = /srv/samba/rh  
valid users = @rh  
read only = no  
browsable = yes  
create mask = 0660
```

`valid users = @rh` est la ligne charnière du projet : elle relie directement le monde réseau (Samba) au groupe Unix `rh` construit dans la Couche 1. Sans cette Couche 1, cette ligne ne protégerait rien.

Avant tout démarrage : validation obligatoire de la syntaxe avec `testparm` — jamais démarrer un service réseau sur une config non vérifiée.

Preuve par le test

Une config qui « a l'air bonne » ne vaut rien sans validation réelle. Test depuis un vrai client SMB (`smbclient`), pas juste en local sur le serveur.

Test 1 — accès légitime

```
$ smbclient //localhost/rh -U rh_awa
smb: \> ls
  awa1.txt                N      0  ...
smb: \> mkdir test_reseau
smb: \> ls
  test_reseau             D      0  ...  ✓ écriture confirmée
```

Test 2 — isolation entre départements

```
$ smbclient //localhost/it -U rh_awa
tree connect failed: NT_STATUS_ACCESS_DENIED  ✓ refus confirmé
```

Les deux résultats sont aussi importants l'un que l'autre : prouver que l'accès fonctionne **et** prouver qu'il se bloque exactement où il doit se bloquer.

Ce que je retiens vraiment

- **La sécurité, c'est des couches qui s'appuient les unes sur les autres** — cassez la fondation Linux, et toute la config Samba au-dessus devient décorative.
- **Un dossier a un seul groupe propriétaire** — au-delà, il faut des ACL, pas du bricolage.
- **Toujours tester ce qu'on croit avoir compris** — plusieurs suppositions logiques (mot de passe unique par service, chemin de fichier, syntaxe bash) se sont révélées fausses à l'exécution, jamais sur le papier.
- **Automatiser dès que ça se répète** — une boucle bash pour 10 comptes, pas 10 commandes collées à la main.
- **Vérifier avant de croire** — un éditeur qui semble « vide » dans un terminal Docker n'est pas un fichier vide : toujours confirmer avec un second outil (`cat -A`) avant d'agir.

"On ne comprend pas vraiment une permission tant qu'on n'a pas vu, de ses yeux, la commande échouer pour la mauvaise raison qu'on croyait."